

LEBANESE
INTERNATIONAL
UNIVERSITY



جامعة اللبنانية الدولية
الكلية اللبنانية
العلوم والتكنولوجيا

School Of Arts and Sciences
Department of Computer Science/Information
Technology

CSCI390
Web Programming

Week 04 : Lecture 02

CSS Tables

- The **border-collapse** specifies whether the browser should control the appearance of the adjacent borders that touch each other or whether each cell should maintain its style.
- The **border-spacing** specifies the width that should appear between table cells.
- The **caption-side** captions are presented in the <caption> element. By default, these are rendered above the table in the document. You use the *caption-side* property to control the placement of the table caption.
- The **empty-cells** specifies whether the border should be shown if a cell is empty.
- The **table-layout** allows browsers to speed up layout of a table by using the first width properties it comes across for the rest of a column rather than having to load the whole table before rendering it.

CSS Tables

The border-collapse Property

```
<style type = "text/css">
  table.one {border-collapse:collapse;}
  table.two {border-collapse:separate;}

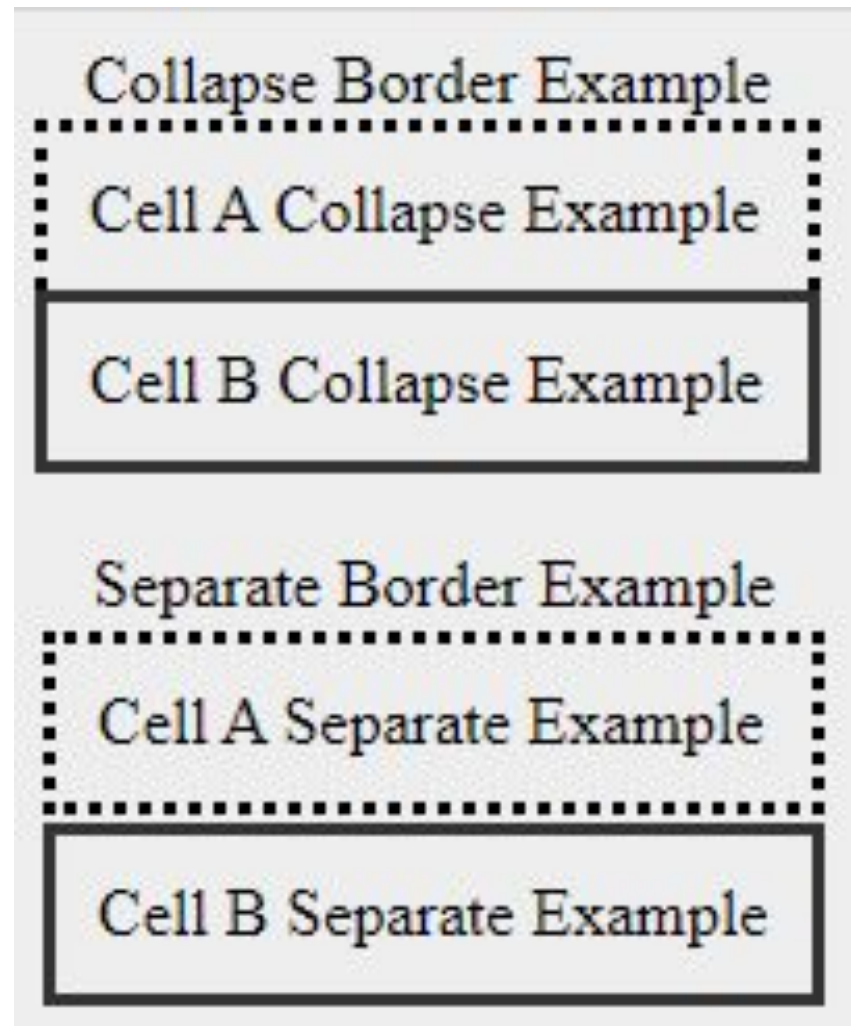
  td.a {
    border-style:dotted;
    border-width:3px;
    border-color:#000000;
    padding: 10px;
  }
  td.b {
    border-style:solid;
    border-width:3px;
    border-color:#333333;
    padding:10px;
  }
</style>
```

```
<body>
  <table class = "one">
    <caption>Collapse Border Example</caption>
    <tr><td class = "a"> Cell A Collapse Example</td></tr>
    <tr><td class = "b"> Cell B Collapse Example</td></tr>
  </table>
  <br />

  <table class = "two">
    <caption>Separate Border Example</caption>
    <tr><td class = "a"> Cell A Separate Example</td></tr>
    <tr><td class = "b"> Cell B Separate Example</td></tr>
  </table>
</body>
```

CSS Tables

The border-collapse Property



```
<style type = "text/css">
  table.one {border-collapse:collapse;}
  table.two {border-collapse:separate;}

  td.a {
    border-style:dotted;
    border-width:3px;
    border-color:#000000;
    padding: 10px;
  }
  td.b {
    border-style:solid;
    border-width:3px;
    border-color:#333333;
    padding:10px;
  }
</style>
```

CSS Tables

The border-spacing Property

```
<style type = "text/css">
  table.one {
    border-collapse: separate;
    width: 400px;
    border-spacing: 10px;
  }
  table.two {
    border-collapse: separate;
    width: 400px;
    border-spacing: 10px 50px;
  }
</style>
```

Separate Border Example with border-spacing

Cell A Collapse Example

Cell B Collapse Example

Separate Border Example with border-spacing

Cell A Separate Example

Cell B Separate Example

CSS Tables

The caption-side Property

```
<style type = "text/css">
  caption.top {caption-side:top}
  caption.bottom {caption-side:bottom}
  caption.left {caption-side:left}
  caption.right {caption-side:right}
</style>
```

```
<table style = "width:400px; border:1px solid black;">
  <caption class = "top">
    This caption will appear at the top
  </caption>
  <tr><td > Cell A</td></tr>
  <tr><td > Cell B</td></tr>
</table>
```

```
<table style = "width:400px; border:1px solid black;">
  <caption class = "bottom">
    This caption will appear at the bottom
  </caption>
  <tr><td > Cell A</td></tr>
  <tr><td > Cell B</td></tr>
</table>
<br />
```

CSS Tables

The caption-side Property

This caption will appear at the top	
Cell A	
Cell B	
Cell A	
Cell B	
This caption will appear at the bottom	
This caption will appear at the left	
Cell A	
Cell B	
This caption will appear at the right	
Cell A	
Cell B	

CSS Tables

The empty-cells Property

The empty-cells property indicates whether a cell without any content should have a border displayed.

```
<style type = "text/css">
  table.empty {
    width:350px;
    border-collapse:separate;
    empty-cells:hide;
  }
  td.empty {
    padding:5px;
    border-style:solid;
    border-width:1px;
    border-color:#999999;
  }
</style>
```

```
<table class = "empty">
  <tr>
    <th></th>
    <th>Title one</th>
    <th>Title two</th>
  </tr>

  <tr>
    <th>Row Title</th>
    <td class = "empty">value</td>
    <td class = "empty">value</td>
  </tr>

  <tr>
    <th>Row Title</th>
    <td class = "empty">value</td>
    <td class = "empty"></td>
  </tr>
</table>
```

	Title one	Title two
Row Title	value	value
Row Title	value	

CSS Tables

Table Width and Height

```
table {  
    width: 100%;  
}
```

```
th {  
    height: 70px;  
}
```

CSS Tables

Horizontal Alignment

- The `text-align` property sets the horizontal alignment (like left, right, or center) of the content in `<th>` or `<td>`.
- By default, the content of `<th>` elements are center-aligned and the content of `<td>` elements are left-aligned.
- To center-align the content of `<td>` elements as well, use `text-align: center;`

```
td {  
    text-align: center;  
}
```

CSS Tables

Vertical Alignment

- The `vertical-align` property sets the vertical alignment (like top, bottom, or middle) of the content in `<th>` or `<td>`.
- By default, the vertical alignment of the content in a table is middle (for both `<th>` and `<td>` elements).
- The following example sets the vertical text alignment to bottom for `<td>` elements:

```
td {  
    height: 50px;  
    vertical-align: bottom;  
}
```

CSS Tables

Table Styling

```
th, td {  
  border-bottom: 1px solid #ddd;  
}
```

```
tr:hover {  
  background-color: #f5f5f5;  
}
```

For zebra-striped tables, use the `nth-child()` selector and add a `background-color` to all even (or odd) table rows:

```
tr:nth-child(even) {  
  
  background-color: #f2f2f2;  
}
```

CSS Flexbox Layout Module

CSS Flexbox is short for the CSS Flexible Box Layout module.

Flexbox is a layout model for arranging items (horizontally or vertically) within a container, in a flexible and responsive way.

The Flexible Box Layout Module, makes it easier to design flexible responsive layout structure without using float or positioning.

```
<style>
.flex-container { display: flex;
  flex-wrap: nowrap;
background-color: DodgerBlue;
}
```

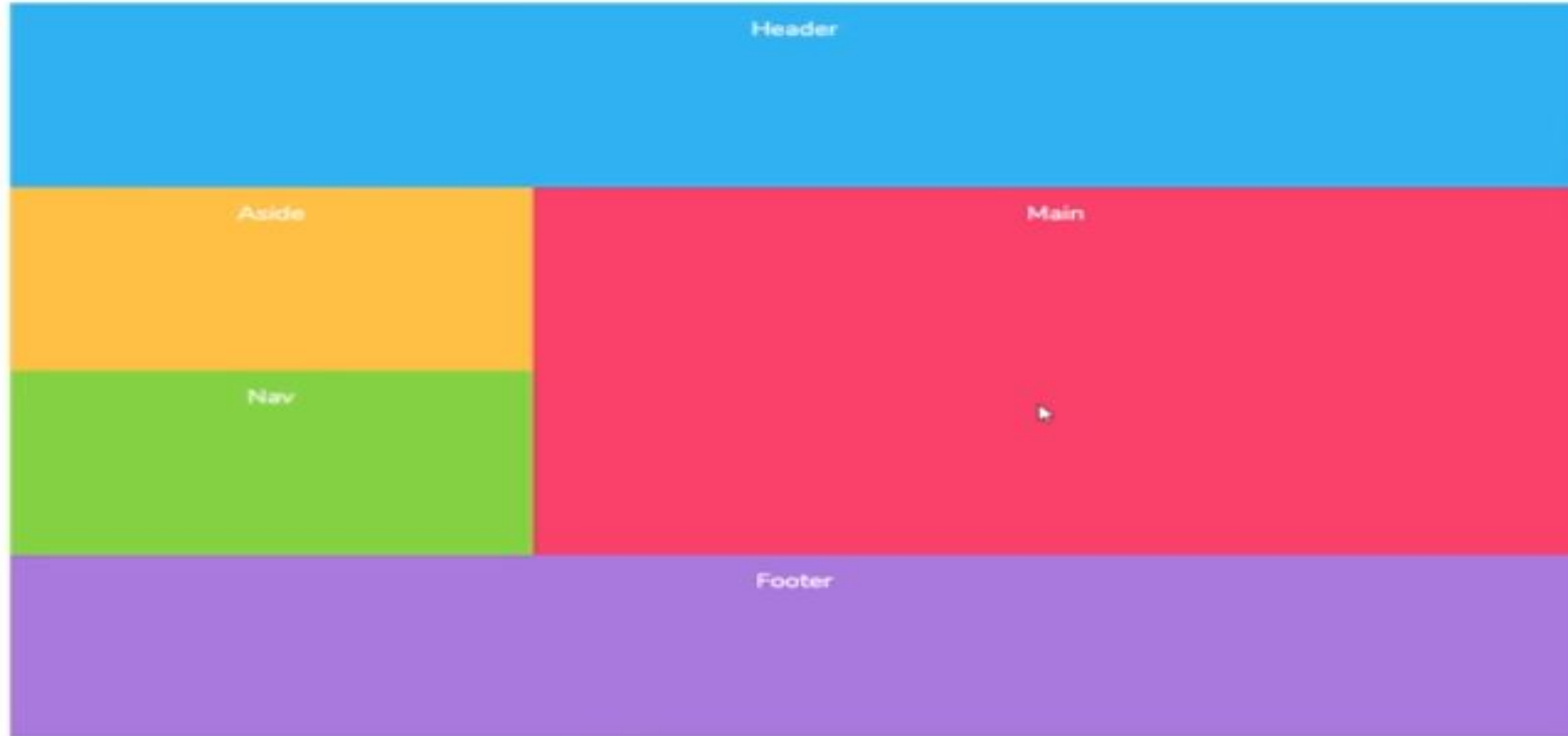
```
.flex-container div {
  background-color: #f1f1f1;
width: 100px; margin: 10px;
text-align: center;
line-height: 75px;
font-size: 30px;
}
</style>
```

```
<body>
<h1>Flexible Boxes</h1>
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
</div>

<p>Try to resize the browser window.</p>
<p>A container with "flex-wrap: nowrap;" will never wrap its
items.</p>
<p><strong>Note:</strong> Flexbox is not supported in Internet
Explorer 10 or earlier versions.</p>

</body>
```

CSS Grid



CSS Grid Container

- To make an HTML element behave as a grid container, you have to set the display property to *grid* or *inline-grid*.
- Grid containers consist of grid items, placed inside columns and rows.
- The grid-template-columns property defines the number of columns in your grid layout, and it can define the width of each column.
- If you want your grid layout to contain 4 columns, specify the width of the 4 columns, or "auto" if all columns should have the same width.

```
<style>
.grid-container {display: grid;
grid-template-columns: auto auto auto
auto;
grid-gap: 10px;
background-color: #2196F3;
padding: 10px;
}
.grid-container > div {
background-color: rgba(255, 255, 255, 0.8);
text-align: center;
padding: 20px 0;
font-size: 30px;
}
</style>
```

```
<body>
<h1>grid-template-columns</h1>
<p>You can use the
<em>grid-template-columns</em> property to
specify the number of columns in your grid
layout.</p>
<div class="grid-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
</div>
</body>
```

CSS Grid Layout Module

- The CSS Grid Layout Module offers a grid-based layout system, with rows and columns, making it easier to design web pages without having to use floats and positioning.

```
<style>
.item1 { grid-area: header; }
.item2 { grid-area: menu; }
.item3 { grid-area: main; }
.item4 { grid-area: right; }
.item5 { grid-area: footer; }
.grid-container {
display: grid; grid-template-areas:
'header header header header header header'
'menu main main main right right'
'menu footer footer footer footer footer';
grid-gap: 10px; background-color: #2196F3; padding: 10px;
}
.grid-container > div {
background-color: rgba(255, 255, 255, 0.8);
text-align: center;
padding: 20px 0;
font-size: 30px;
}
</style>
```

```
<body>
```

```
<h1>Grid Layout</h1>
```

```
<p>This grid layout contains six columns and
three rows:</p>
```

```
<div class="grid-container">
  <div class="item1">Header</div>
  <div class="item2">Menu</div>
  <div class="item3">Main</div>
  <div class="item4">Right</div>
  <div class="item5">Footer</div>
</div>
```

```
</body>
```